



南開大學
Nankai University

并行程序设计实验报告

ANN 近似最近邻搜索的 Pthread / OpenMP 并行优化

Flat / SQ / PQ / FastScan / IVF / IVF-PQ / HNSW

2026 年 5 月 10 日

目录

摘要	2
1 问题定义与目标	2
2 系统设计	2
2.1 目录与模块	2
2.2 并行框架	3
3 算法实现	3
3.1 Flat-SIMD	3
3.2 SQ-SIMD	4
3.3 PQ-SIMD 与 FastScan	4
3.4 IVF-SIMD	4
3.5 IVF-PQ-SIMD	4
3.6 HNSW 进阶	4
4 实验设计	5
4.1 平台与数据集	5
4.2 结果文件与参数	5
5 实验结果与分析	6
5.1 核心矩阵最优结果	6
5.2 Inter-query 与 Intra-query	6
5.3 Pthread 与 OpenMP 调度	6
5.4 Flat 线程伸缩性	7
5.5 IVF / IVF-PQ 的 Latency-Recall Trade-off	7
5.6 HNSW 图搜索	8
6 要求覆盖情况	8
7 总结	9
AI 辅助说明	9

摘要

本实验围绕 DEEP100K 上的近似最近邻搜索 (ANNS) 实现了一个 header-only 的多算法并行框架。系统以 SIMD 距离计算为底层公共内核, 在此基础上实现 Flat-SIMD、SQ-SIMD、PQ-SIMD、PQ-FastScan、IVF-SIMD、IVF-PQ-SIMD 与 HNSW 图索引, 并分别给出 Pthread 与 OpenMP 版本。Pthread 侧实现了 static、dynamic 与 pool 三类调度; 并行粒度覆盖 query 级并行 (inter-query) 与单 query 内部的 base/list 划分 (intra-query)。实验在本机 i9-13900H 平台完成 278 组核心矩阵结果, 并补充 15 组 IVF / IVF-PQ 的 `nprobe` 与构建模式 trade-off 结果。

实验结果显示, query 级并行是本数据集和批量查询场景下最稳健的并行粒度。Flat、SQ、PQ、FastScan、IVF 与 IVF-PQ 的最优点均来自 inter-query; 其中 Flat、SQ、PQ、FastScan、IVF 与 IVF-PQ 的本地最优延迟分别为 298.95、86.53、73.06、125.81、104.67 与 27.44 μs 。Pthread dynamic 在多数规则扫描类算法上表现最好, Pthread pool 在 IVF inter-query 上略优, 说明线程创建开销、尾部不均衡与 inverted list 不均匀性都会影响最终表现。HNSW 的内部并行更复杂: 原始 baseline 延迟最低但召回较低, 多入口点和边划分能提升召回, 但同步和路径依赖使其不适合作为默认最快路径。

关键词: ANN, Pthread, OpenMP, SIMD, IVF, IVF-PQ, HNSW, FastScan, 负载均衡

1 问题定义与目标

给定基向量集合

$$X = \{x_1, x_2, \dots, x_N\} \subset \mathbb{R}^d$$

以及查询向量 $q \in \mathbb{R}^d$, k 近邻搜索希望找到 X 中与 q 距离最近的 k 个向量。本文使用的距离形式沿用课程框架中的内积距离:

$$\delta(x, q) = 1 - \sum_{i=1}^d x_i q_i.$$

近似最近邻搜索不要求每次完全返回真实 top- k , 因此用 Recall@ k 衡量准确率:

$$\text{Recall@}k = \frac{|KNN(q) \cap KANN(q)|}{k}.$$

本次作业的核心目标不是单独做 SIMD, 而是在已经具备 SIMD 距离内核的基础上, 系统实现 Pthread / OpenMP 多线程并行, 并从以下方面进行评估:

1. 对 Flat、SQ、PQ、FastScan、IVF、IVF-PQ 与 HNSW 进行统一实现和测试;
2. 比较 Pthread static、dynamic、pool 与 OpenMP 的任务划分和调度开销;
3. 比较 inter-query 与 intra-query 两种并行粒度;
4. 分析线程数、负载均衡、top- k reduce、cache / 内存访问和负优化现象;
5. 对 IVF / IVF-PQ 给出 `nprobe` 与构建模式对 latency-recall 的影响。

2 系统设计

2.1 目录与模块

实现按功能拆分为公共 SIMD 内核、Pthread 并行层、OpenMP 并行层、IVF 系列、HNSW 系列和独立 main 程序。所有新增算法均采用 header-only 方式, 满足课程框架只编译 `main.cc` 的约束。

模块	作用
simd/	统一存放 Flat、SQ、PQ、FastScan 的 SIMD 距离计算和查表扫描内核，例如 flat_scan_simd.h、pq_fastscan_simd.h。
pthread/	Flat、SQ、PQ、FastScan 的 Pthread 并行实现，以及公共线程池 thread_pool.h。
omp/	Flat、SQ、PQ、FastScan 的 OpenMP 并行实现。
ivf/	IVF 索引、IVF-SIMD、IVF-PQ 以及对应 Pthread / OpenMP 并行搜索。
hnsw/	基于 hnswlib 的多入口点、边划分、Layer0 划分与 IVF-HNSW 嵌套搜索。
mains/	71 个可独立编译的实验入口，用于不同算法、调度策略和并行粒度的测试。
results/local/	本地实验原始输出文件。每个文件包含 label、Recall@10 与平均查询延迟。

表 1 代码模块划分

2.2 并行框架

本文实现了两类并行粒度。

Inter-query 并行。每个线程处理一批完整 query。线程间不共享搜索状态，只写入结果数组中的不同位置，因此同步开销最低。对于批量查询，inter-query 通常能获得最好的吞吐和延迟表现。

Intra-query 并行。对单条 query，将 base 向量集合、候选集合或 IVF inverted list 分给多个线程。每个线程维护局部 top- k 或 top- p 堆，最后由主线程 merge。这种方式适合单 query 延迟优化，但会引入局部堆维护、线程同步和 top- k reduce 成本。

Pthread 侧实现三种调度方式：

- static: 静态等分任务，适合 Flat 这类计算量规则的扫描；
- dynamic: 用原子计数器动态抢任务，适合 query 耗时或 list 长度不均匀的场景；
- pool: 预创建线程池和任务队列，减少反复创建 / 销毁线程的开销。

OpenMP 侧主要使用 `parallel for`，query 级循环使用 dynamic schedule，base/list 扫描使用局部堆加串行 merge。图 1 展示整体数据流。

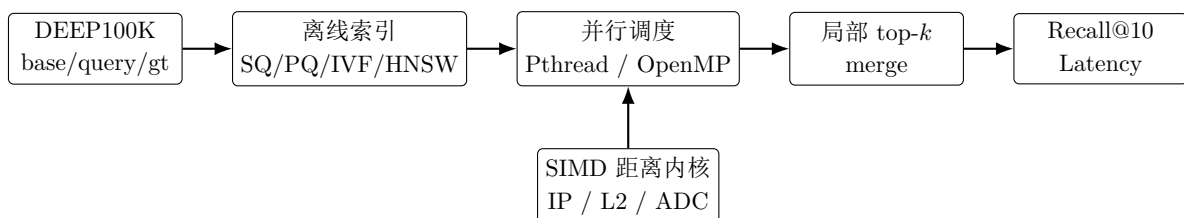


图 1 ANN 多线程并行框架

3 算法实现

3.1 Flat-SIMD

Flat 是精确全量扫描。每个 query 需要访问全部 N 个 base 向量并维护 top- k 堆。距离计算复用 simd/flat_scan_simd.h，在 x86 上使用 AVX2，在 ARM 上保留 NEON 路径。

Flat 的计算量规则，因此理论上 static 划分也应较强。但在本机 Windows 混合核调度环境下，dynamic query 抢任务可以减轻尾部等待和系统噪声，实际取得最优结果。Intra-query 版本需要每个线程维护局部堆并在单 query 结束后 merge，因此固定开销显著。

3.2 SQ-SIMD

SQ 将 float32 向量逐维量化为 uint8，查询阶段采用两阶段流程：第一阶段在 uint8 空间快速粗排得到 top- p 候选，第二阶段对候选使用原始 float32 内积进行 rerank。本文在 Pthread 与 OpenMP 中都实现了 query 级整条流水线并行，以及单 query 内粗排阶段 base 划分并行。由于精排候选集合较小，精排阶段主要保持单线程执行，避免额外同步开销。

3.3 PQ-SIMD 与 FastScan

PQ 将 96 维向量切分为 $M = 8$ 个子空间，每个子空间训练 256 个中心。查询时对每个子空间构建 LUT，并用 ADC 对 base 的 PQ code 查表累加。Pthread / OpenMP 版本分别对 query 循环或 base code 扫描进行并行化。

FastScan 是 PQ 的进一步压缩实现，使用 4-bit code 和更小 LUT，将查表过程转为寄存器内 shuffle。FastScan 的 ADC 阶段更偏 memory-bound，线程数过多可能更早触及带宽上限。因此本文重点比较 inter-query 与 intra-query 的差异，并在报告中保留负优化结果。

3.4 IVF-SIMD

IVF 在离线阶段用 KMeans 将 base 聚为 `nlist` 个簇。查询时先计算 query 到各簇中心的距离，选择最近的 `nprobe` 个簇，再扫描这些 inverted list 并用 SIMD 原始距离 rerank。实现中保留簇内连续存储信息，减少精排阶段的随机访问。

IVF 粗排计算量小，真正适合并行的是精排阶段。由于不同 inverted list 长度不均匀，dynamic 和 pool 调度更有意义。本文实现：

- IVF-SIMD baseline;
- query 级 Pthread / OpenMP 并行;
- 单 query 内按 list 或候选段划分并行;
- `nprobe` sweep，用于展示 latency-recall trade-off。

3.5 IVF-PQ-SIMD

IVF-PQ 将 IVF 的候选缩减与 PQ 的压缩距离计算叠加。本文实现两种构建方式：

1. global-PQ-first: 先对所有 base 做全局 PQ 编码，再构建 IVF;
2. IVF-local-PQ: 先构建 IVF，再在簇内维护局部 PQ 编码。

查询时先 IVF 粗排选择 `nprobe` 个簇，再在选中簇内用 PQ ADC 得到候选并 rerank。由于 PQ 显著减少单次距离计算量，IVF-PQ 更容易转为内存访问和调度开销主导，线程数并非越多越好。

3.6 HNSW 进阶

HNSW 基于 hnsplib 实现。本文不修改 hnsplib 源码，而是通过封装访问底层图结构，实现以下并行策略：

- baseline: 原始 `searchKnn`;
- 多入口点并行: 多个线程从不同入口出发, 最后 merge top- k ;
- 边划分并行: 并行计算当前节点邻居距离;
- Layer0 点划分: 按节点 ID 范围切分底层图扫描;
- IVF-HNSW 嵌套: 先 IVF 粗排, 再并行搜索多个小 HNSW。

图搜索路径依赖强, 内部并行容易被同步和候选队列更新开销抵消, 因此 HNSW 部分重点如实分析正优化与负优化。

4 实验设计

4.1 平台与数据集

本地实验平台为 Intel Core i9-13900H, 14 核 20 逻辑线程; 编译器为 MSYS2 GCC 14.2.0。核心编译选项为:

```
-O2 -mavx2 -mfma -fopenmp -lpthread -std=c++17 -I.
```

数据集为 DEEP100K: 100K 条 96 维 base 向量, 10K 条 query, ground truth 为 top-100。所有正式本地实验取前 2000 条 query, $k = 10$ 。报告中的平均延迟均为查询阶段平均延迟, 不包含离线索引构建时间。

4.2 结果文件与参数

核心矩阵结果位于 `results/local/`, 汇总文件为 `results/local_summary.csv`。补充 trade-off 结果为 `results/local_tradeoff.csv`。表 2 给出有效结果数量。

算法族	核心矩阵结果	补充 trade-off	线程集合
Flat	40	0	1,2,4,8,16
SQ	32	0	1,4,8,16
PQ	40	0	1,2,4,8,16
FastScan	32	0	1,4,8,16
IVF	45	5	1,2,4,8,16
IVF-PQ	45	10	1,2,4,8,16
HNSW	44	0	1,4,8,16
合计	278	15	—

表 2 本地实验结果覆盖范围

核心矩阵的固定参数如下:

- SQ/PQ 取 $p = 100$, FastScan 取 $p = 1000$;
- IVF 取 `nlist=16,nprobe=4`;
- IVF-PQ 取 `nlist=16,nprobe=4,p=100,mode=global`;
- HNSW 取 `ef=50`, IVF-HNSW 嵌套取 `nlist=16,nprobe=4`。

5 实验结果与分析

5.1 核心矩阵最优结果

表 3 汇总每个算法族的最优时延配置。加速比以同一算法族最快的 1 线程结果为基准，因此反映多线程扩展能力，而不是相对最朴素串行程序的总加速。

算法族	最优变体	线程	Recall@10	延迟 / μs	相对最快 1 线程
Flat	flat_pthread_dynamic_inter	16	0.99995	298.95	10.67×
SQ	sq_pthread_dynamic_inter	16	0.99995	86.53	8.53×
PQ	pq_pthread_dynamic_inter	16	0.70860	73.06	9.37×
FastScan	fastscan_pthread_dynamic_inter	16	0.97525	125.81	10.51×
IVF	ivf_pthread_pool_inter	16	0.96225	104.67	7.25×
IVF-PQ	ivfpq_pthread_dynamic_inter	16	0.69180	27.44	8.71×
HNSW	hnsw_baseline	1	0.96865	65.48	1.00×

表 3 各算法族核心矩阵最优点

Flat 与 SQ 都能达到接近精确搜索的召回。PQ 和 IVF-PQ 在 $p = 100$ 下速度很快但 recall 较低，说明 PQ 的候选数需要与召回目标联动调节。FastScan 在 $p = 1000$ 下保持 0.97525 的 recall，并且 inter-query 多线程能把平均延迟压到 125.81 μs 。IVF 的 recall 与速度介于 Flat 和 PQ 系列之间， $nprobe=4$ 时达到 0.96225。

5.2 Inter-query 与 Intra-query

表 4 对比每个算法族最优的 inter-query 与 intra-query 结果。除 HNSW 外，所有扫描类算法的最佳结果均来自 inter-query。

算法族	最优 inter 延迟 / μs	inter 线程	最优 intra 延迟 / μs	intra 线程
Flat	298.95	16	1726.09	16
SQ	86.53	16	704.31	4
PQ	73.06	16	528.97	2
FastScan	125.81	16	1322.76	1
IVF	104.67	16	628.62	2
IVF-PQ	27.44	16	410.31	4

表 4 query 级并行与 query 内并行的最优结果对比

原因主要有三点。第一，DEEP100K 实验使用 2000 条 query，query 级并行天然有足够任务数，线程间几乎没有共享状态。第二，intra-query 需要多个局部 top- k 堆和最终 merge，固定开销在每条 query 上重复出现。第三，PQ、FastScan 和 IVF-PQ 的单 query 内部计算已经被量化压缩，继续切分会更快遇到 cache、内存带宽和同步瓶颈。

5.3 Pthread 与 OpenMP 调度

表 5 给出 16 线程下若干代表性 inter-query 结果。Pthread dynamic 在多数算法中优于 OpenMP，Pthread pool 在 IVF 中表现最好。

算法	OMP inter	Pthread static	Pthread dynamic	Pthread pool
Flat	341.55	340.96	298.95	347.41
SQ	118.30	126.12	86.53	92.42
PQ	119.30	75.67	73.06	252.03
FastScan	139.26	137.33	125.81	126.73
IVF	113.90	132.08	116.94	104.67
IVF-PQ	106.84	29.81	27.44	55.08

表 5 16 线程 inter-query 下的调度策略对比, 单位为 μs

Dynamic 的优势来自更好的尾部负载均衡。虽然每次抢任务需要原子操作, 但 query 数量足够多, 单个 query 的计算量也远大于一次 `fetch_add`。IVF 的 pool 结果略优, 说明 inverted list 扫描和索引对象访问使线程复用收益更明显。OpenMP 写法简洁, 但在本机运行环境中调度常数和 runtime 行为不总是最优。

5.4 Flat 线程伸缩性

Flat 的 inter-query 线程伸缩性如表 6。从 1 到 16 线程, 三类 Pthread 变体均有明显加速, 但 8 到 16 线程收益已经下降, 说明全量扫描逐渐接近内存带宽上限。

变体	1 线程	2 线程	4 线程	8 线程	16 线程
OMP inter	6769.48	1737.01	634.21	433.47	341.55
Pthread static inter	5160.16	2214.15	675.36	436.45	340.96
Pthread dynamic inter	3312.96	1407.33	563.03	388.78	298.95
Pthread pool inter	3188.90	1542.36	578.00	398.82	347.41

表 6 Flat inter-query 不同线程数延迟, 单位为 μs

5.5 IVF / IVF-PQ 的 Latency-Recall Trade-off

表 7 展示 IVF 在 16 线程 Pthread pool inter-query 下调整 `nprobe` 的结果。随着 `nprobe` 增大, 扫描簇数增加, recall 单调上升, 延迟也总体上升。`nprobe=4` 是本地核心矩阵中使用的折中点, Recall@10 为 0.96225。

<code>nprobe</code>	Recall@10	延迟 / μs	说明
1	0.73665	61.47	最低延迟, 召回不足
2	0.87870	93.41	召回明显提升
4	0.96225	131.54	较好折中点
8	0.99430	210.35	接近精确召回
16	0.99995	455.89	几乎退化为更大范围扫描

表 7 IVF 的 `nprobe` trade-off, `nlist=16`, 16 线程

表 8 对比 IVF-PQ 的 global-PQ-first 与 IVF-local-PQ 两种构建方式。Local-PQ 在本地数据上 recall 更高, 尤其在低 `nprobe` 下优势明显, 因为每个簇内单独量化能减少局部误差; 但簇内 PQ 结构更复杂, 构建与内存布局成本更高。Global-PQ-first 实现简单、存储统一, 但在 $p = 100$ 时 recall 上限明显较低。

nprobe	Global recall	Global 延迟 / μs	Local recall	Local 延迟 / μs
1	0.56240	49.73	0.65885	12.74
2	0.64840	59.39	0.76480	21.08
4	0.69180	85.08	0.82035	28.99
8	0.70725	47.24	0.83760	48.15
16	0.70870	81.43	0.84010	92.65

表 8 IVF-PQ 两种构建方式的 nprobe trade-off, p=100, 16 线程

需要说明的是, IVF-PQ 的多线程结果存在一定运行噪声, 尤其是在延迟已经降到几十微秒时, 线程调度、缓存状态和系统后台负载都会造成非单调波动。因此表 8 更适合观察 recall 趋势和两种构建方式差异, 而非把每一个延迟点解释为严格单调曲线。

5.6 HNSW 图搜索

HNSW 的结果见表 9。baseline 最快, 但 Recall@10 只有 0.96865。多入口点和边划分能提升召回, 其中 `hnsw_edge_omp` 在 0.99915 recall 下延迟为 390.91 μs , `hnsw_multi_entry_omp` 在 4 线程下达到 1.00000 recall, 延迟为 464.85 μs 。Layer0 点划分接近底层全扫描, 召回高但延迟很大。

变体	线程	Recall@10	延迟 / μs 与分析
<code>hnsw_baseline</code>	1	0.96865	65.48, 最快, 但召回低于高精度要求
<code>hnsw_ivf_nested_static</code>	16	0.94705	221.90, 速度较快但召回下降
<code>hnsw_edge_omp</code>	1	0.99915	390.91, 高召回代表点
<code>hnsw_multi_entry_omp</code>	4	1.00000	464.85, 多入口点提高覆盖面
<code>hnsw_layer0_static</code>	4	0.99995	3400.41, 接近底层点划分扫描, 代价过高

表 9 HNSW 代表性结果

图搜索与向量扫描的差别在于搜索路径强依赖当前候选队列。线程越多, 并不意味着能均匀拆分出更多独立工作。边划分在邻居数较小时无法摊薄同步成本, 多入口点更像用额外计算换 recall, Layer0 点划分则牺牲了 HNSW 本身减少访问点的优势。因此 HNSW 的并行优化目标应是 recall-latency 折中, 而不是单纯追求线程加速比。

6 要求覆盖情况

表 10 对照作业要求列出完成情况。

要求项	完成情况
Pthread 基础并行	Flat、SQ、PQ、FastScan、IVF、IVF-PQ 均实现 static / dynamic / pool 的 inter 和 intra 版本; HNSW 实现 static / dynamic / pool 相关策略。
OpenMP 基础并行	对 Flat、SQ、PQ、FastScan、IVF、IVF-PQ 和 HNSW 主要策略实现 OpenMP 版本, 编译使用 <code>-fopenmp</code> 。
与 SIMD 结合	所有扫描类方法统一调用 <code>simd/</code> 下的 AVX2 / NEON 距离或查表内核。
Flat-SIMD 多线程	完成 query 级并行与 base 划分并行, 比较不同线程数和调度方式。

PQ-SIMD 多线程	完成 PQ ADC 的 inter / intra 并行，分析 LUT 查表后 memory-bound 对扩展性的影响。
FastScan 进阶	实现 4-bit PQ FastScan 的 Pthread / OpenMP 并行版本，并给出补充矩阵。
IVF-SIMD baseline 与多线程	实现 IVF 索引、SIMD 搜索、Pthread / OpenMP 并行，并补充 nprobe trade-off。
IVF-PQ-SIMD baseline 与多线程	实现 IVF-PQ、两种构建模式、Pthread / OpenMP 并行，并比较 global/local 构建差异。
HNSW 进阶	基于 hnswlib 实现 baseline、多入口点、边划分、Layer0 点划分与 IVF-HNSW 嵌套结构。
性能测试与报告分析	本地收集 278 组核心矩阵结果和 15 组补充 trade-off 结果，分析线程数、调度、负载均衡、cache / 内存访问和负优化。

表 10 作业要求覆盖情况

7 总结

本实验从单一 SIMD 内核扩展为完整 ANN 并行检索框架。最重要的结论是：在批量 query 场景中，query 级并行比单 query 内切分更稳健。它避免了局部 top- k merge 和频繁同步，并能自然复用 SIMD 内核。Pthread dynamic 对尾部负载和系统调度噪声更宽容，线程池在 IVF 这类索引结构复用场景下也有优势。OpenMP 开发成本低，但 runtime 调度不一定总能取得最低延迟。

从算法角度看，Flat 提供高召回但受内存带宽限制；SQ 在本实验中取得近乎无损 recall 和较低延迟；PQ 与 IVF-PQ 速度很快但需要更大的 p 或更好的构建方式提高 recall；FastScan 说明数据布局和查表方式会改变瓶颈；IVF 通过 nprobe 提供清晰的 latency-recall 旋钮；HNSW 则体现了图搜索并行化的挑战，内部并行更多是为高召回服务，而不是默认提升速度。

后续可以继续完善三点：第一，在鲲鹏平台用 test.sh 2 1 复跑完整矩阵，补齐 ARM NEON 与 x86 AVX2 的多线程对比；第二，对 SQ/PQ/FastScan 做更系统的 p sweep，形成更完整的 latency-recall 曲线；第三，对 HNSW 尝试更细粒度的共享候选队列和无锁 visited 标记，验证是否能降低图搜索并行的同步成本。

AI 辅助说明

本项目使用 Codex 辅助完成代码结构梳理、实验脚本编排、结果汇总和报告一致性检查。所有性能数据均来自本地程序输出文件，未使用 AI 生成或伪造实验数据。关键结果可由 results/local/、results/local_summary.csv 和 results/local_tradeoff.csv 复核。